



Smart Contract Audit

cMATRA Surrender Pool Validator

Prepared for: Flux Point Studios

Date: April 14, 2026

Version: 2.0

Classification: Public

Language: Aiken 1.1.21 / Plutus V3 / stdlib 2.2.0

Scope: On-chain spend validator + off-chain transaction building + co-signing flow

ALL FINDINGS REMEDIATED

1. Executive Summary

A comprehensive security audit was conducted on the cMATRA surrender pool validator — a parameterized Plutus V3 spend script written in Aiken that governs a pool of approximately 818 million cMATRA on the Cardano blockchain during a 6-month public redemption window.

The audit analyzed **20 distinct attack vectors** across the on-chain validator, the Aiken standard library, the off-chain transaction-building service, the two-party co-signing flow, and Cardano-specific address manipulation attacks. The analysis was informed by published vulnerability research from Vacuumlabs, MLabs, Plutonomicon, Anastasia Labs, and real-world audit findings from Cardano DeFi protocols.

Six off-chain findings were identified (4 medium, 2 low) — all remediated. No critical or high severity vulnerabilities were found. The on-chain validator logic is correct. One design-level observation (no on-chain value preservation) is documented as an intentional architectural decision with appropriate mitigations.

Overall Assessment: The surrender pool validator is correctly implemented and suitable for mainnet deployment. Authorization and time-lock controls are enforced without exploitable gaps. All off-chain findings have been remediated with defense-in-depth measures including pool UTxO reservation, dedicated collateral, strict datum validation, mandatory transaction preflight, and frankenaddress protections.

2. Validator Overview

2.1 Architecture

The surrender pool validator is a parameterized spend validator. Pool UTxOs at the script address hold cMATRA tokens with Void (unit) inline datums. The validator enforces two invariants for every pool UTxO spend:

1. **Dual-signer authorization:** Two designated administrator verification key hashes must both appear in the transaction's required signers. The keys are intended to reside on separate infrastructure, eliminating single-point-of-failure risk.
2. **Time control:** The transaction's validity range must fall entirely within the correct window — before the deadline for surrender processing, or after the deadline for pool withdrawal.

2.2 Parameters

Parameter	Type	Description
<code>admin_pkh_1</code>	Blake2b-224 hash (28 bytes)	Primary admin key hash (Server A), baked in at compile time
<code>admin_pkh_2</code>	Blake2b-224 hash (28 bytes)	Co-signer key hash (Server B), baked in at compile time
<code>deadline</code>	POSIX milliseconds (Int)	End of the surrender window, baked in at compile time

2.3 Spending Paths

Redeemer	Conditions	Purpose
<code>ProcessSurrender</code>	Both admins signed AND validity range entirely before deadline	Process a user's asset surrender during the open window
<code>AdminWithdraw</code>	Both admins signed AND validity range entirely after deadline	Reclaim remaining pool balance after the window closes

2.4 Validator Source

```
validator surrender_pool(admin_pkh_1: VerificationKeyHash, admin_pkh_2:
VerificationKeyHash, deadline: Int) {
  spend(datum: Option<Void>, redeemer: SurrenderAction, _own_ref: OutputReference, tx:
Transaction) {
    expect Some(_) = datum
    let admin_1_signed = list.has(tx.extra_signatories, admin_pkh_1)
    let admin_2_signed = list.has(tx.extra_signatories, admin_pkh_2)
    let both_signed = admin_1_signed && admin_2_signed

    when redeemer is {
      ProcessSurrender ->
        both_signed && is_entirely_before(tx.validity_range, deadline)
      AdminWithdraw ->
        both_signed && is_entirely_after(tx.validity_range, deadline)
    }
  }

  else(_) { fail }
}
```

3. Findings Summary

#	Severity	Finding	Location	Status
1	Medium	Pool UTxO contention — concurrent builds target same UTxO	Off-chain	Remediated
2	Medium	Collateral auto-selection from admin wallet	Off-chain	Remediated
3	Medium	UTxO datum spoofing — pool discovery accepted non-Void datums	Off-chain	Remediated
4	Medium	No mandatory preflight before transaction submission	Off-chain	Remediated
5	Low	Datum-less UTxO grieving in pool discovery	Off-chain	Remediated
6	Low	Frankenaddress: user address and configuration validation	Off-chain	Remediated
7	Design	No on-chain value preservation (admin discretion by design)	On-chain	Accepted
8– 20	Secure	Double satisfaction, deadline boundaries, unbounded ranges, authorization, datum robustness, non-spend rejection, reference scripts, parameter immutability, PKH collision, bait-and-switch, reference input abuse, token dust, transaction metadata	On-chain + Off-chain	Verified

4. Detailed Findings

Finding 1 — Pool UTxO Contention [Medium]

In the eUTxO model, each pool UTxO can only be consumed by one transaction per block. The transaction-building service selected the largest available pool UTxO for every request. When multiple surrender requests arrived concurrently, all targeted the same UTxO, causing all but one to fail at submission. An attacker could amplify this by sending rapid requests to exhaust pool UTxO availability.

Remediation: A pool UTxO reservation system was implemented. When a pool UTxO is selected for a build, it is marked as reserved for 2 minutes (sufficient for the user to review and sign). Concurrent builds automatically skip reserved UTxOs and select the next available one. Reservations are released on build failure, successful submission, or TTL expiry.

Finding 2 — Collateral Auto-Selection [Medium]

The transaction builder allowed pycardano to auto-select collateral from the administrator's wallet UTxOs. In the event of a phase-2 validation failure, the collateral is consumed as a penalty (150% of the transaction fee on mainnet). Auto-selection could choose a high-value UTxO, resulting in disproportionate collateral loss.

Remediation: The service now supports an explicit collateral UTxO configuration. When set, the builder uses a dedicated small-value UTxO (5 ADA recommended) as collateral, bounding maximum loss on any phase-2 failure.

Finding 3 — UTxO Datum Spoofing [Medium]

Pool discovery accepted any UTxO with any datum present, rather than verifying the datum was specifically `Void ( Constr(0, []) )`. An attacker could send cMATRA to the script address with a non-Void datum (e.g., `Constr(1, [0xdeadbeef])`), which would pass the off-chain filter but fail the on-chain validator's type deserialization, causing unnecessary transaction failures.

Remediation: Pool discovery now validates that each UTxO's inline datum matches the exact `Void` structure (`constructor: 0, fields: []`). UTxOs with datum hashes only (no inline datum), missing datums, or non-Void datums are rejected. Applied to all three pool discovery locations.

Finding 4 — No Mandatory Preflight [Medium]

Transactions were built and returned to the user without first verifying they would pass on-chain script evaluation. If the transaction contained an error (stale UTxO, misconfigured script parameters,

incorrect datum), the user would discover this only after signing and submitting — at which point collateral is consumed on phase-2 failure.

Remediation: The build endpoint now runs `evaluate_tx` against the live ledger as a mandatory step before returning the transaction CBOR. If the Plutus script evaluation fails, the transaction is rejected and the pool UTxO reservation is released. This catches errors before any collateral is at risk.

Finding 5 — Datum-less UTxO Griefing [Low]

Anyone can send cMATRA to the script address without a datum. The on-chain validator correctly rejects these UTxOs, but the off-chain code previously selected them, causing avoidable build failures. Remediated by filtering for datum presence (now subsumed by Finding 3's stricter Void CBOR validation).

Finding 6 — Frankenaddress Exposure [Low]

Three address surfaces were analyzed for frankenaddress (mangled staking credential) risk. The pool script address uses enterprise-type encoding (no staking credential) and is secure. The user address endpoint now rejects script-payment credentials. The quarantine address is validated at startup with a warning if a staking credential is present.

Finding 7 — No On-Chain Value Preservation [Design]

The validator does not enforce output constraints. The administrator has full discretion over fund routing. This is a deliberate design decision — rate enforcement occurs off-chain. If the admin signing key were compromised, the attacker could drain the pool. Mitigated through operational key security, API authentication, rate limiting, and mandatory preflight evaluation.

5. Attack Vector Analysis

5.1 Double Satisfaction

SECURE

When multiple pool UTxOs are consumed in a single transaction, the validator executes once per input against the same transaction context. Since only the administrator can construct valid transactions and modifying any output invalidates the signature, multi-input exploitation is not feasible.

5.2 Deadline Boundary Semantics

SECURE

The Aiken stdlib functions `is_entirely_before` and `is_entirely_after` were traced through their source. A clean dead zone exists at the exact deadline: transactions whose validity range includes the deadline moment are rejected by both paths.

Validity Range	ProcessSurrender	AdminWithdraw
Upper bound = deadline - 1 (inclusive)	Allowed	Rejected
Includes deadline (inclusive either bound)	Rejected	Rejected
Lower bound = deadline + 1 (inclusive)	Rejected	Allowed
Unbounded (-inf, +inf)	Rejected	Rejected

5.3 Dual-Signer Authorization Model

SECURE

The validator requires both `admin_pkh_1` and `admin_pkh_2` in `tx.extra_signatories`. The Cardano ledger enforces that every PKH in `required_signers` has a corresponding VK witness. A transaction signed by only one admin key is rejected. This eliminates single-point-of-failure risk: an attacker who compromises one server cannot drain the pool without also compromising the other.

5.4 Datum Robustness & Deserialization

SECURE

`expect Some(_) = datum` rejects `None` datums (Plutus V3 allows spend without datum via CIP-69). Aiken fails safely on type deserialization mismatch — non-Void datums cause a hard script abort, not unexpected behavior.

5.5 Non-Spend Purpose Rejection (Conway-Era)

SECURE

The `else(_) { fail }` catch-all rejects all six non-spend Plutus V3 script purposes: Mint, Withdraw, Publish (certificates), Vote, and Propose. This includes Conway-era governance actions (DRep voting, proposal guardrails). The wildcard pattern is forward-compatible with future Plutus versions.

5.6 Bait-and-Switch / Transaction Modification

SECURE

The admin signs the transaction body hash. The user can only add their VK witness — they cannot modify inputs, outputs, fees, or redeemers. Any modification invalidates the admin signature. The submit endpoint verifies the transaction hash matches the one built by the service, preventing substitution of a different transaction.

5.7 Reference Input Abuse (CIP-31)

SECURE

The validator does not read reference inputs. Pool UTxOs can be referenced without triggering the spending validator, but this is benign — pool balances are public knowledge and datums are Void.

5.8 Token Dust / Value Spam

SECURE

Anyone can send arbitrary native tokens to the script address. Pool discovery filters by the specific cMATRA unit and Void datum, ignoring junk UTxOs. Token dust creates new UTxOs at the address but cannot modify existing pool UTxOs.

5.9 Reference Script & Parameter Security

SECURE

Parameters are baked into the CBOR bytecode via `aiken blueprint apply`. The script address is derived from the compiled hash. Different parameters produce a different address. Deterministic builds allow third-party verification of the deployed script against the published source.

6. Frankenaddress Analysis

A *frankenaddress* combines one entity's payment credential with another's staking credential, allowing the attacker to earn delegation rewards on ADA locked at that address. Three surfaces were analyzed:

- **Pool address:** Enterprise-type (no staking credential). Canonical-only queries. **Secure.**
- **User address:** Script-payment credentials now rejected. **Remediated.**
- **Quarantine address:** Startup validation warns if staking credential present. **Remediated.**

7. Industry Research

This audit was cross-referenced against published vulnerability research and real-world audit findings from the Cardano ecosystem to ensure comprehensive coverage.

Source	Vectors Checked
Vacuumlabs Vulnerability Series (#1–#6)	Double satisfaction, trust-no-UTxO, time handling, token security
MLabs Common Vulnerabilities Guide	11 named vulnerability classes including UTxO contention, unbounded datum, arbitrary datum injection, other-redeemer bypass
Plutonomicon Vulnerability Catalog	Infinite mint, unbounded value, parameterization verification, cheap spam
Anastasia Labs Audit Patterns	Hardcoded reference input index, stake validator pattern, design patterns
Tweag / Minswap Post-Mortem	Duplicated pool NFT via minting policy flaw (not applicable to this validator)
Adamant Security Research	Mangled addresses, staking credential hijacking, multi-sig concerns
June 2024 Cardano DDoS Analysis	REWARD-type script abuse, reference script fee asymmetry
CIP-69 (Plutus V3 Signature Unification)	Optional datum in V3 spend scripts, single ScriptContext argument

8. Test Coverage

Source	Tests	Coverage
On-chain	10	Both-admin ProcessSurrender, post-deadline ProcessSurrender (fail), pre-deadline AdminWithdraw (fail), both-admin AdminWithdraw, single-admin-1 ProcessSurrender (fail), single-admin-2 ProcessSurrender (fail), single-admin-1 AdminWithdraw (fail), non-admin ProcessSurrender (fail), non-admin AdminWithdraw (fail), missing datum (fail)
Red-team	10	All on-chain tests + wrong redeemer (fail), fabricated UTxO (fail), single-admin ProcessSurrender (fail), single-admin AdminWithdraw (fail), both-admin happy-path ProcessSurrender, both-admin happy-path AdminWithdraw. Executed against live preprod deployment.
Preprod	9 stages	Full lifecycle: wallet creation, cMATRA minting, pool deployment, user surrender, red-team attacks, admin withdrawal. 9/9 stages passed.

9. Deployment Recommendations

- **Admin key separation:** The dual-signer model requires both admin keys on separate infrastructure (Server A and Server B). Neither key should be accessible from the other server's environment. Generate key pairs independently and deploy each to its respective server only.
- **Compiler version pinning:** Pin Aiken compiler version in CI/CD. A different compiler version may produce a different script hash.
- **Deterministic build verification:** Publish the exact script hash and build instructions so that third parties can independently verify the deployed validator matches the audited source.
- **Enterprise-type addresses:** Use addresses without staking credentials for both the script address and quarantine address.
- **Dedicated collateral:** Configure a dedicated small-value collateral UTxO to bound maximum loss.

10. Conclusion

The cMATRA surrender pool validator has undergone a comprehensive security review informed by the Cardano ecosystem's published vulnerability research and real-world audit findings. Twenty

attack vectors were analyzed across the on-chain validator, the Aiken standard library, the off-chain transaction service, the co-signing flow, and Cardano-specific address manipulation patterns.

Six findings were identified in off-chain code (4 medium, 2 low) — all have been remediated. The on-chain validator logic is correct: authorization and time-lock invariants are enforced without exploitable gaps, the Aiken stdlib interval functions exhibit clean boundary semantics, and the `else(_) { fail }` pattern correctly rejects all non-spend purposes including Conway-era governance actions.

Assessment: All identified vulnerabilities have been resolved. The cMATRA surrender pool validator is correctly implemented and suitable for mainnet deployment.